

Project Report
on
**SMART X-RAY LUGGAGE DETECTION
SYSTEM**

Submitted in the partial fulfillment for the requirements
For the award of the degree of



Bachelor of Engineering
in
Electronics and Telecommunication Engineering

By

Yash Fanse	B-843
Harshal Dhavan	B-845
Laxman Birajdar	B-846
Chirag Pingale	B-847

Under the Supervision of

Dr. Viplav Soliv

Academic Year 2025-2026

MCT
MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI
(Permanently Affiliated to University of Mumbai)
Department of Electronics and Telecommunication Engineering)

CERTIFICATE

This is to certify that the project entitled “**SMART X-RAY LUGGAGE DETECTION SYSTEM**” is a bona fide work of **Yash Fanse** (Roll no. B843), **Harshal Dhavan** (Roll no. B845), **Laxman Birajdar** (Roll no. B846), **Chirag Pingale** (Roll no. B847) under the supervision of **Dr. Viplav Soliv**, submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **Bachelor of Engineering in Electronics and Telecommunication Engineering**.

Dr. Viplav Soliv
Supervisor

Dr. Sanjay D. Deshmukh
Head of Department

Dr. Sanjay U. Bokade
Principal

REPORT APPROVAL FOR B.E.

This project report entitled “**SMART X-RAY LUGGAGE DETECTION SYSTEM**” by **Yash Fanse, Harshal Dhavan, Laxman Birajdar, Chirag Pingale** is approved for the degree of **Bachelor of Engineering in Electronics and Telecommunication Engineering** from University of Mumbai, in academic year 2025-26.

Examiners:

1. Internal: _____

2. External: _____

Date: _____

Place: Mumbai

Declaration

This document is our declaration of how the content of this document is our original work, expressed in our words. In addition, in case we have used the words/ideas of any other individual, we have made the necessary citation for the same. This is further to state that we have followed all academic standards of integrity and honesty and have not presented any false data, facts, idea or source. We know well that violating the above mentioned principle is going to attract disciplinary actions from our institute. Furthermore, legal action may also follow if there has been plagiarism from any source without due permission.

Yash Fanse (B-843) ()

Harshal Dhavan (B-845) ()

Laxman Birajdar (B-846) ()

Chirag Pingale (B-847) ()

Date:

Place: Mumbai

Acknowledgments

It is a matter of great pleasure and pride to present this project on **SMART X-RAY LUGGAGE DETECTION SYSTEM**. The authors express their sincere gratitude to the Principal, **Dr. Sanjay U. Bokade**, and the Head of the Department, **Dr. Sanjay D. Deshmukh**, for providing us with the necessary facilities and academic environment.

The authors are grateful to our project guide, **Dr. Viplav Soliv**, Assistant Professor, Department of Electronics and Telecommunication Engineering, for his continuous guidance, valuable suggestions, and technical support throughout the development of this project. His encouragement and insights greatly contributed to the successful completion of this work.

We also extend our appreciation to all faculty members, colleagues, and peers who provided constructive feedback and motivation during the design, implementation, and testing phases of the system. Special thanks to the Department of Electronics and Telecommunication Engineering, Rajiv Gandhi Institute of Technology, Andheri, Mumbai, for the laboratory resources and support provided.

Abstract

The fast pace of development of the airline industry and the public transport sector has resulted in an increasing need for effective and robust baggage security inspection systems. The traditional X-ray scanning systems require human intervention, which poses problems such as operator fatigue and slow reaction time to potential threats. This project introduces the concept of the **SMART X-RAY LUGGAGE DETECTION SYSTEM**.

The object detection algorithm that is designed using YOLOv8 is responsible for detecting prohibited objects, including guns, knives, scissors, pliers, and wrenches, in real-time through video footage of luggage placed on the conveyor belt. Multiple frames detection is utilized to minimize any false alarm detection cases. In case a prohibited object is detected, the STM32F103C8T6 microcontroller takes charge in controlling the entire system process of alerting and rejecting the prohibited object.

Moreover, this system allows for cloud-based surveillance via the FastAPI framework and the PostgreSQL database. Experimental results indicate that the YOLOv8m model produces an mAP@50 score of 0.91 at about 25 FPS and has an end-to-end detection to response latency of 150–200 ms, making it appropriate for deployment as an efficient and economical security system for luggage bags.

Contents

1	Introduction	2
2	Literature Survey	3
3	Problem Statement and Objectives	4
3.1	Problem Statement	4
3.2	Objectives	4
4	Methodology	6
4.1	System Design and Architecture	6
4.2	Hardware Layer	7
4.3	Software Layer	8
4.4	AI Layer	9
4.5	Operating Modes	10
4.6	Cloud Integration and Monitoring	10
4.7	Power Management	10
5	Requirements	11
5.1	Hardware Requirements	11
5.2	Software Requirements	12
5.3	Network and Communication Requirements	13
5.4	Security and Privacy Requirements	13
6	Components and Technologies	14
6.1	AI Detection Engine: YOLOv8	14
6.1.1	Dataset	14
6.1.2	Multi-Frame FSM Confirmation	14
6.2	Embedded Controller: STM32F103C8T6	15
6.3	Backend Technologies	15
6.3.1	Local Detection Software	15
6.3.2	Cloud Backend: FastAPI and PostgreSQL	15
6.3.3	Web Dashboard	16
6.4	Mechanical Automation Subsystem	16

7	Implementation Details	17
7.1	AI Model Training	17
7.2	Real-Time Detection Pipeline	17
7.3	Embedded Firmware	17
7.4	Cloud Integration	18
7.5	Local Dashboard	19
8	Results and Discussion	20
8.1	Model Training Performance	20
8.2	Detection Accuracy Analysis	20
8.3	System Latency Analysis	20
8.4	Hardware Response Validation	21
8.5	Real-Time System Performance	22
8.6	Cloud Monitoring Evaluation	22
8.7	Cloud Performance Analysis	22
8.8	Challenges and Solutions	22
8.8.1	False Positives in Detection	22
8.8.2	Inference Speed on CPU	23
8.8.3	UART Communication Reliability	24
8.8.4	Object Overlap and Cluttered Luggage	24
9	Conclusion	25
10	Future Scope	26
10.1	Edge AI Deployment	26
10.2	Multi-Camera Integration	26
10.3	Exploration of Advanced Detection Architectures	26
10.4	Realistic X-Ray Image Simulation	26
10.5	Enhanced Cloud Platform and Analytics	27
10.6	Improved False Positive Suppression	27
10.7	Scalability Through Cloud Deployment	27
	Bibliography	28

List of Figures

4.1	System Architecture Block Diagram	6
8.1	Confusion Matrix of YOLOv8m Detection Model	21
8.2	Cloud Dashboard Showing Safe State Detection	23
8.4	Cloud Dashboard Showing Danger State Detection	23

List of Tables

5.1	Hardware Requirements	11
5.2	Software Requirements	12
8.1	Class-wise Detection Performance Metrics	20
8.2	System End-to-End Latency Breakdown	21

Chapter 1

Introduction

With the increasing in the number of local traveling and using public transport, there is a huge need for fast luggage scanning systems. Traditional luggage scanning systems mainly involves human inspections who manually view the X-ray scans on monitor. Luggage scanning systems are used in airports and other highly secured places to detect illegal items like guns, knives, and dangerous equipment. This raises problems like tiredness, different interpretations, slow decision-making, and insecurity issues.

While significant advancement have been made in AI enabled threat detection systems, many of the existing systems still does not incorporate the detections within the hardware systems that carry out the necessary actions upon detection of the threat. Simply identifying a potential hazard is not enough, there must be an automated process for responding to such hazards.

In order to mitigate such issues, the current study introduces a smart X-ray luggage threat detection system based on AI detection and hardware automation. While existing systems rely only on the ability to detect threats, this new system goes beyond this and responds accordingly by removing the suspicious items. Real-world applications in airports, metro stations.

Chapter 2

Literature Survey

1. **J. Redmon, S. Divvala, R. Girshick, and A. Farhadi**, *You Only Look Once: Unified, Real-Time Object Detection*, CVPR, 2016.

This pioneering paper proposed the YOLO model that reformulated the problem of object detection as a simple regression task from image pixels to bounding box locations and class probabilities. The idea allowed detecting objects in real time with high frame rates and served as a base for the design of YOLOv8. The single-stage nature of the method is particularly important for our use case.

2. **S. Akcay, M.E. Kundegorski, M. Devereux, and T.P. Breckon**, *Transfer Learning Using Convolutional Neural Networks for Object Classification within X-Ray Baggage Security Imagery*, IEEE ICIP, 2016.

It was found by Akcay et al. that transfer learning with CNNs is a highly effective approach to classifying objects that cannot be brought aboard with one's carry-on luggage. This paper has a way for future research which will be conducted in the area of using deep learning in the area of security imagery classification.

3. **C. Miao et al.**, *SIXray: A Large-Scale Security Inspection X-Ray Benchmark for Prohibited Item Discovery in Overlapping Images*, IEEE/CVF CVPR, 2019.

Miao et al. developed the SIXray dataset, a massive security dataset based on X-rays, that was purposefully constructed to facilitate the recognition of prohibited objects within overlapping and complicated bag images.

4. **C.Y. Wang, A. Bochkovskiy, and H.Y.M. Liao**, *YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors*, IEEE/CVF CVPR, 2023.

The YOLOv7 detector was the latest and fastest real-time detector at its point of release using trainable Bag of Freebies that enhances the model's accuracy without raising the cost of computation.

5. **A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi**, *Internet of Things for Smart Cities*, IEEE Internet of Things Journal, vol. 1, no. 1, pp. 22–32, 2014.

The concepts of cloud connectivity, remote monitoring, and real-time data synchronization mentioned in this paper were used to develop the cloud monitoring component of the system architecture, based on WebSocket communications and FastAPI services.

Chapter 3

Problem Statement and Objectives

3.1 Problem Statement

X-ray scanning technology is frequently used at airports and other dangerous transit points in several nations to screen for any type of forbidden goods in the luggage. There is a single significant disadvantage of the current procedures for screening luggage by X-ray scans: the entire procedure depends extensively on individuals who are skilled at analyzing X-rays.

Fatigue induced by lengthy work hours causes distraction, leading to the possibility that something hazardous may be overlooked. Human error can lead to inconsistent screening since each person is unique. There is also a delay between when the threat is recognized and when an action is taken.

Research into AI-based approaches for detecting existing technologies has been done for precision. Still, there is no automation to guarantee that the threatening item is isolated. Detection of the threat without implementing any automation to isolate the object cannot be regarded a suitable solution.

It is imperative to develop an automated system that can detect threats in real-time using artificial intelligence and perform additional activities such as conveying isolation, hardware rejection, alerting, remote monitoring, etc.

3.2 Objectives

1. **Detect prohibited items in X-ray luggage images using AI:** Deploy YOLOv8m for identifying five classes of prohibited items – guns, knives, scissors, pliers, and wrenches from live video feeds.
2. **Implement multi-frame confirmation to reduce false positives:** Ensure that the detection itself is validated through an FSM model, that uses sequence of 12 frames for consistency in detections leading to a threat warning.
3. **Automate conveyor control using an embedded microcontroller:** Deploy the STM32F103C8T6 Bluepill board to halt the operation of the conveyor belt and activate the solenoid actuator to physically isolate the threat.
4. **Provide real-time multi-level alerts:** Raise multiple alerts using sound (buzzer), light (LEDs, LCD), and software (dashboard) alarms upon detection.

5. **Enable cloud-based remote monitoring:** store detection events along with snapshots at a FastAPI-backed cloud PostgreSQL server accessible through a protected web-based dashboard.
6. **Achieve real-time performance with low latency:** Ensure latency between the detection and action initiation phases within 150-200 ms to make it practically feasible for use with a conveyor belt setup.

Chapter 4

Methodology

4.1 System Design and Architecture

The design of the proposed system is an intelligent and flexible security system utilizing artificial intelligence, embedded systems, mechanics, and cloud computing. In the proposed system, video frames are first obtained from a web camera fitted above the conveyor belt. The input data will be analyzed in real-time by the YOLOv8 algorithm on a computer having a GPU. On the other hand, the output data will appear on the screen and transferred to the cloud database. This will trigger a command via UART to activate the STM32 microcontroller to control the remaining components of the security system.

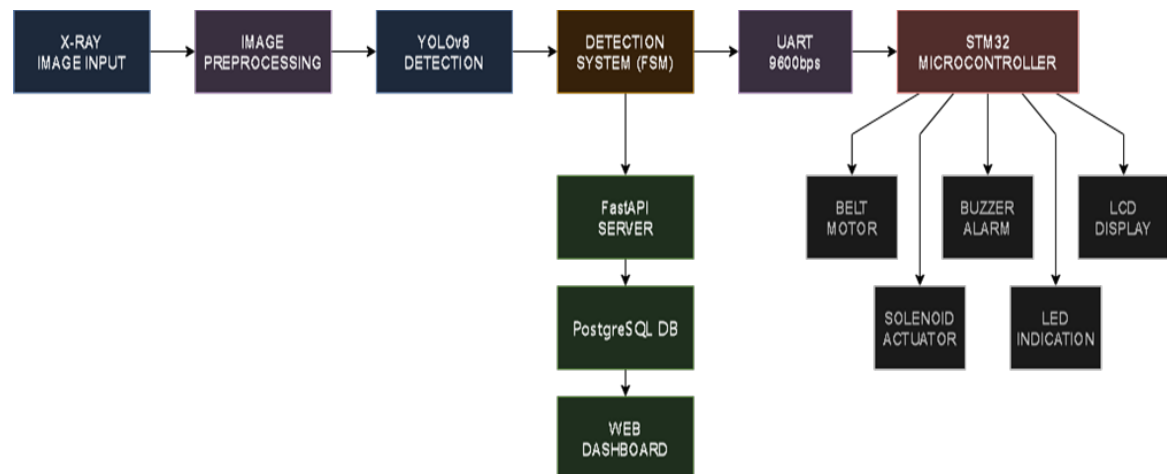


Figure 4.1: System Architecture Block Diagram

4.2 Hardware Layer

The hardware layer comprises all physical equipment required for image capture, embedded control, and real-time alert generation.

1. **GPU-Enabled Computer:** Webcam is placed above the conveyor, recording the video frame of the luggage with a frame rate of 30 fps per second continuously.
2. **USB Camera:** Webcam is placed above the conveyor belt, with recording the video frame of the luggages with a frame rate of around 30 fps per second continuously..
3. **STM32F103C8T6 Bluepill Microcontroller:** Serves as an embedded microcontroller that receives UART commands from the threat detection system and controls output actuators like the relay on the conveyor, solenoid, buzzer, and LED indicators..
4. **Conveyor Belt (12V DC Motor):** The model consists of a conveyer belt of size approximately 4x26 inches that is driven by a 12V DC motor running at speed controlled by PWM and direction controlled by DPST.
5. **Solenoid Actuator (12V):** It activates the solenoid to actuate the threats physically from the conveyer path.
6. **Green and Red LEDs:** The Green LED will remain active when in Safe Mode and will be activated when it is in Danger Mode.
7. **Buzzer:** Works at a frequency of 1 kHz in Danger Mode once the threat is detected to provide an audible signal.
8. **LCD Display:** Indicates real-time status either of “SYSTEM SAFE” when everything is okay or “DANGER: THREAT DETECTED.”
9. **SMPS Power Supply:** Provides the required power by converting 230V AC to 12V DC. It uses a buck converter to supply 5V DC.

4.3 Software Layer

This layer links the object recognition module to all parts of the system. Thus, communication between the AI engine, controller hardware, monitoring interface, and backend cloud becomes possible in real time.

1. **Detection Software (Python + OpenCV):** Captures video using `cv2.VideoCapture`, performs preprocessing (brightness adjustment, sharpening, resizing to 640×640), runs YOLOv8m inference, and applies a 12-frame FSM confirmation buffer before generating a threat decision.
2. **Local Dashboard (Flask + pywebview):** Desktop-based GUI offering real-time video stream with bounding boxes, confidence levels, system information, and detections logged.
3. **Embedded Firmware (STM32 Arduino IDE):** Constantly listens to UART for command signals — “0” indicates SAFE MODE while “1” denotes DANGER MODE — and operates all GPIO outputs appropriately.
4. **Cloud Backend (FastAPI + PostgreSQL):** Detection events are transmitted through WebSocket connections, saved in PostgreSQL database alongside threat images and logs, then served to the web-based GUI via secured API endpoints.
5. **Web Dashboard:** Remotely accessible to view current detection information, history of detected events, visualization of threats, and system condition checks.

4.4 AI Layer

Analyzing of the video streams feed, also identifying the prohibited items with accuracy within the AI layer.

1. **Model Architecture:** For the selection of an appropriate model, YOLOv8m was chosen due to its medium performance and balance between the detection accuracy and inference time.
2. **Dataset:** An open-source airport X-ray dataset that contains 8,295 labeled images (training - 5,806; validation - 1,660; testing - 829).
3. **Detected Classes:** Object categories: Gun, Knife, Scissors, Pliers, Wrench.
4. **Training Parameters:**
 - Framework → Ultralytics YOLOv8
 - Image Size → 640×640
 - Epochs → 35
 - GPU → NVIDIA RTX 2060
 - Confidence Threshold → 0.80
 - Python → 3.10 (Anaconda environment)
5. **Multi-Frame FSM Confirmation:** The identified objects should be stable across 12 frames to be recognized as a threat.

4.5 Operating Modes

The STM32 embedded microcontroller controls the entire system in three different modes depending on the UART signals received from the detection module.

1. **Safe Mode:** The green LED is switched on, the buzzer is inactive, and the LCD displays "SYSTEM SAFE". The conveyor belt runs continues and the solenoid stays deactivated.
2. **Danger Mode:** The conveyor belt is stopped, the solenoid pushes out the detected object, the buzzer rings at 1 KHz, the red LED glows, and the LCD shows "DANGER: THREAT DETECTED".
3. **Emergency Stop Mode:** Activated by pressing the push-button switch manually. All actions stop instantly, and only after resetting can it be restarted.

4.6 Cloud Integration and Monitoring

The detections are sent to the cloud backend using WebSockets protocol. The FastAPI server is responsible for storing threat images along with their timestamps and class into the PostgreSQL database. JWT-based security ensures that only authorized users can access the dashboard. Web Dashboard allows real-time monitoring, logging, visualization, and manipulation of system remotely via the web/mobile browser.

4.7 Power Management

Power in the system operates using the dual rail principle. The first rail consists of 230V AC mains which are stepped down to 12V DC using the SMPS module to power the motor and solenoid actuators. The second rail consists of 5V regulated supply using the Buck Converter to power the STM32, LCD, LED, Buzzer and Relay circuitry.

Chapter 5

Requirements

5.1 Hardware Requirements

Component	Description
USB Camera	1080p resolution, 30 FPS, mounted above conveyor belt
GPU-Enabled Computer	NVIDIA RTX 2060, 6GB VRAM, CUDA-enabled for deep learning inference
STM32F103C8T6 (Bluepill)	Dual-core ARM Cortex-M3, 72 MHz, 20 KB SRAM, UART communication
12V DC Motor	Drives conveyor belt (4×26 inches), PWM speed control
Solenoid Actuator	12V, relay-controlled, physically ejects flagged luggage
Green LED	Standard 5mm LED, 2.0V forward voltage, Safe Mode indicator
Red LED	Standard 5mm LED, 2.0V forward voltage, Danger Mode indicator
Buzzer	Active piezo buzzer, 5V, 1 kHz activation on threat detection
LCD Display	16×2 character LCD for real-time system status display
SMPS Module	230V AC to 12V DC power conversion
Buck Converter	12V to 5V regulation for logic-level components
Resistors	220 Ω resistors for LED current limiting

Table 5.1: Hardware Requirements

5.2 Software Requirements

Software / Library	Version / Details
Python	3.10 (Anaconda environment)
Ultralytics YOLOv8	Latest stable release
PyTorch	2.0+ with CUDA support
OpenCV (cv2)	4.x for video capture and image processing
Flask	Python micro-framework for local dashboard
FastAPI	Cloud backend API framework
PostgreSQL	Cloud database for event and snapshot storage
pywebview	Desktop GUI wrapper for local monitoring app
Arduino IDE	2.0+ for STM32 firmware development
STM32 Board Package	STM32duino by STMicroelectronics
Operating System	Windows 11 / Ubuntu 22.04

Table 5.2: Software Requirements

5.3 Network and Communication Requirements

1. **UART Serial Communication:** The detection computer communicates with the STM32 microcontroller via UART at 9600 bps using a USB-to-serial adapter.
2. **WebSocket Communication:** Detection events are transmitted to the FastAPI cloud backend via WebSocket for real-time data synchronization.
3. **FastAPI Server:** Hosted on a cloud platform, exposes secured REST API endpoints for dashboard access and data retrieval.
4. **JWT Authentication:** All cloud dashboard and API access is protected with JSON Web Token (JWT) based authentication.
5. **Internet Connectivity:** Required on the detection computer for cloud event synchronization and remote dashboard access.

5.4 Security and Privacy Requirements

- All local processing operates within the detection computer; video frames are not permanently stored except as threat snapshots.
- The cloud backend is secured with JWT-based authentication, preventing unauthorized dashboard access.
- Threat event logs and snapshots are stored in the PostgreSQL database with access restricted to the only who is authorized administrator.
- UART communication between the detection system and STM32 is local and cannot be intercepted remotely.

Chapter 6

Components and Technologies

6.1 AI Detection Engine: YOLOv8

The detection algorithm works with the YOLOv8m (medium) architecture, which was developed by Ultralytics. The YOLOv8 algorithms are the latest versions of the YOLO architecture family, where the features extraction procedure is optimized using C2f backbone architecture and anchor-free detection head. In this regard, the training procedure for the YOLOv8m algorithm was performed using a post-training algorithm with a customized set of X-ray images of prohibited objects.

In the course of performing the inference step, the model assesses all the frames from the video and then generates boundaries for each detected object, along with a textual representation of the class label of the detected object, and the confidence score associated with the prediction. Predictions whose confidence scores fall below 0.80 are not included to prevent any false-positive errors. The inference speed is 25 FPS on the NVIDIA RTX 2060 GPU.

6.1.1 Dataset

The training data set comprises 8,295 labeled images from a publicly available airport security data set found in Roboflow. The data set is distributed as follows:

- **Training Set:** 5,806 images (70%)
- **Validation Set:** 1,660 images (20%)
- **Testing Set:** 829 images (10%)
- **Classes:** Gun, Knife, Scissors, Pliers, Wrench
- **Image Resolution:** 640×640 pixels
- **Data Augmentation:** Brightness adjustment, rotation, horizontal flipping, and sharpening

6.1.2 Multi-Frame FSM Confirmation

A FSM confirmation buffer of size 12 frames is employed to ensure that the process of detection is stable. The threat will only be detected after the same object category has

been identified consecutively for 12 frames. This helps prevent false triggering because of inaccurate detection.

6.2 Embedded Controller: STM32F103C8T6

The STM32F103C8T6 Bluepill microcontroller serves as the embedded controller for the hardware. It monitors UART serial communication instructions from the sensor module at all times:

- **“0” (SAFE):** Conveyor runs, solenoid off, green LED on, buzzer off, LCD shows safe status.
- **“1” (DANGER):** Conveyor stops, solenoid activates, red LED on, buzzer sounds at 1 kHz, LCD shows danger alert.

The STM32 microcontroller was programmed using the Arduino IDE along with the STM32duino board package for ease in writing C++ programs.

6.3 Backend Technologies

6.3.1 Local Detection Software

The program for local object recognition is programmed in Python through OpenCV. It runs as a desktop application using a graphical user interface made with pywebview. The back-end is locally hosted with a flask-based server providing an API for real-time monitoring dashboard access.

6.3.2 Cloud Backend: FastAPI and PostgreSQL

The cloud-based monitoring application uses FastAPI as a backend framework because of its high speed and capacity for automatic creation of API documentation. The detection data with timestamp, class name, confidence score, and threat image as JPEG format are stored in PostgreSQL relational database. Functions provided by the cloud-based system include:

- **FastAPI Server:** High-performance REST API with automatic OpenAPI documentation.
- **PostgreSQL Database:** Stores all detection events, logs, and metadata.
- **WebSocket Communication:** Real-time event push from the detection system to the cloud server.

- **JWT Authentication:** Secure user authentication for dashboard and API access.
- **Snapshot Storage:** JPEG images of detected threats are stored for audit and review.

6.3.3 Web Dashboard

Real-time monitoring, detection and analysis, event logging, and remote control of the system are provided through the internet dashboard. This dashboard is accessible via any device with an internet connection using either a web browser or a smartphone. The dashboard facilitates monitoring of different security checkpoints.

6.4 Mechanical Automation Subsystem

An experimental conveyor system prototype that is made up of a belt measuring 4×26 inches in dimension was constructed for testing purposes. It mimics the behavior of luggage being conveyed in an airport setting. It is driven by a 12V DC motor, and its speed can be varied via PWM motor controller. When the detecting mechanism is activated, the 12V solenoid actuator releases a load from the conveyor belt. It works on relay logic controlled by the STM32 microcontroller.

Chapter 7

Implementation Details

7.1 AI Model Training

For training YOLOv8m model, 35 epochs were required on the Ultralytics YOLOv8 model with NVIDIA RTX 2060 GPU. Transfer learning on the model using COCO pre-trained weights was used to take advantage of learned features and save time in training. Brightness change, rotation, flip horizontally, and sharpen were some data augmentation strategies used on the X-ray images during training.

7.2 Real-Time Detection Pipeline

Each video frame captured by the USB camera undergoes the following processing steps:

- Frame capture using `cv2.VideoCapture`.
- Preprocessing: brightness adjustment, rotation correction, and sharpening filter application.
- Resizing to 640×640 and normalization.
- YOLOv8m inference with confidence threshold of 0.80.
- FSM-based 12-frame confirmation buffer evaluation.
- Bounding box drawing with class labels and confidence scores on the frame.
- Decision generation: “0” (SAFE) or “1” (DANGER) transmitted via UART to STM32.
- Annotated frame displayed on the local dashboard and event data synchronized to cloud.

7.3 Embedded Firmware

STM32F103C8T6 code was programmed using the Arduino IDE with the STM32duino library. On booting up, all GPIO pins are initialized and the microcontroller listens to the data on UART at 9600 bits per second. After receiving command “1” (DANGER), the microcontroller performs the following process:

- Disable conveyor motor relay (belt stops).
- Activate solenoid relay (object ejected from conveyor).
- Turn on red LED, turn off green LED.
- Activate buzzer at 1 kHz.
- Update LCD display to “DANGER: THREAT DETECTED”.

On receiving command “0” (SAFE), all actuators return to Safe Mode state.

```

1 void loop() {
2   if (Serial.available() > 0) {
3     char cmd = Serial.read();
4     if (cmd == '1') {
5       // DANGER MODE
6       digitalWrite(MOTOR_RELAY, LOW);    // Stop conveyor
7       digitalWrite(SOLENOID, HIGH);      // Eject object
8       digitalWrite(RED_LED, HIGH);
9       digitalWrite(GREEN_LED, LOW);
10      tone(BUZZER_PIN, 1000);             // 1 kHz buzzer
11      lcd.setCursor(0, 0);
12      lcd.print("!! DANGER MODE !!");
13    } else if (cmd == '0') {
14      // SAFE MODE
15      digitalWrite(MOTOR_RELAY, HIGH);    // Run conveyor
16      digitalWrite(SOLENOID, LOW);
17      digitalWrite(RED_LED, LOW);
18      digitalWrite(GREEN_LED, HIGH);
19      noTone(BUZZER_PIN);
20      lcd.setCursor(0, 0);
21      lcd.print("  SYSTEM SAFE  ");
22    }
23  }
24 }

```

Listing 7.1: STM32 Firmware: UART Command Handler

7.4 Cloud Integration

These detection instances will be sent to the FastAPI cloud server through WebSockets. This event includes information like the timestamp, the class name of the detected object, its confidence score, and the image captured from the video frames that pose any danger. Everything is saved inside the PostgreSQL database. The website makes requests to the FastAPI server to receive real-time data updates.

7.5 Local Dashboard

The local monitoring module uses OpenCV to stream real-time annotated videos with bounding boxes drawn in green color for the safe video streams and in red color for the threat detected. The Web view module is used to display current detections, events, and system modes. The Flask module acts as an API server for the dashboard.

Chapter 8

Results and Discussion

8.1 Model Training Performance

The YOLOv8m model is trained using 8,295 X-ray images with annotations for 35 epochs on an NVIDIA RTX 2060 GPU. The model attains mAP@50 of 0.91 with an inference speed of about 25 FPS, considering a confidence score of 0.80.

Class	Precision	Recall	F1-Score	mAP@50
Gun	0.94	0.87	0.90	0.92
Knife	0.93	0.85	0.89	0.91
Pliers	0.92	0.86	0.89	0.90
Scissors	0.91	0.84	0.87	0.89
Wrench	0.92	0.87	0.89	0.91
Overall (Mean)	0.924	0.858	0.888	0.91

Table 8.1: Class-wise Detection Performance Metrics

From the experiment findings, it is evident that the model has been successful in detecting the objects within the X-ray images with impressive precision, since the mAP@50 metric was over 0.89 for each class. The guns and knives have been recognized with the greatest level of certainty because of their distinctive appearance on the radiographic image. Pliers and wrenches were mistaken as similar objects owing to their similar physical appearances.

8.2 Detection Accuracy Analysis

The test performance on the validation set consisting of 829 images showed reliable classification capability. The guns and knives were detected with high accuracy because of their characteristic shapes, but some confusion was seen between pliers and wrenches because of slight similarities. A confidence level of 0.80 and 12 frames' confirmation buffer helped filter out unreliable classifications.

8.3 System Latency Analysis

End-to-end latency from detection to response was calculated by measuring the time span between the verification of threats and the deployment of responses via the actuators under

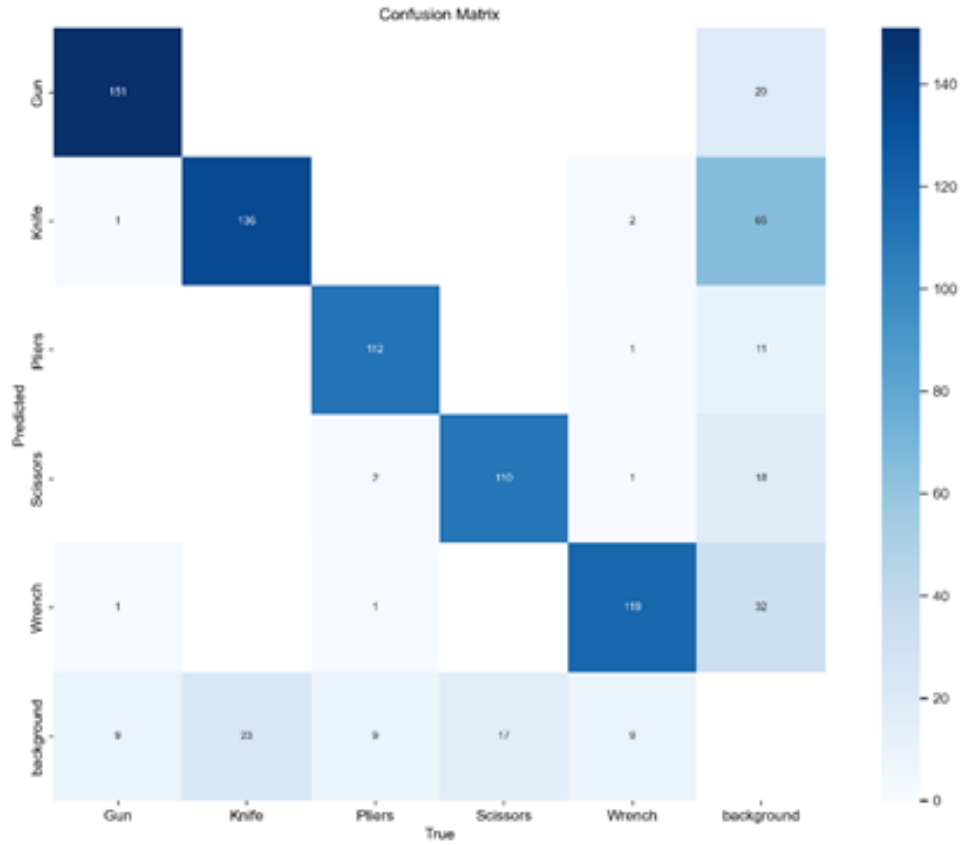


Figure 8.1: Confusion Matrix of YOLOv8m Detection Model

control of STM32 processors. This system delivers a total latency performance of 150-200 milliseconds and is considered adequate enough for real-time application on conveyors.

Stage	Latency (ms)
YOLOv8m Inference (per frame)	~40 ms
12-Frame FSM Confirmation	~480 ms (12 frames @ 25 FPS)
UART Transmission to STM32	<5 ms
STM32 Actuation Response	<80 ms
Total Detection-to-Response	150–200 ms

Table 8.2: System End-to-End Latency Breakdown

8.4 Hardware Response Validation

The automatic mechanism incorporated in the circuit was tested using all the modes available by conducting repeated simulations. In the Safe Mode, the conveyor belt moved as

usual with the green LED signal, but there was no buzzing sound from the buzzer as the alarm did not activate. When the Danger Mode was activated, the conveyor belt stopped moving, the object was discharged via the solenoid, the buzzer made a buzzing sound, and a warning message flashed on the LCD screen.

8.5 Real-Time System Performance

It worked without any failure since it was working in real time without any pause in the motion of the conveyor belt. The frame rates were synchronized at about 25 FPS, instant UART communication to STM32 was realized, and synchronization of everything was ensured in all cases.

8.6 Cloud Monitoring Evaluation

The cloud synchronization module was capable of relaying the images obtained from the detection process, the labels assigned, confidence scores, and timestamps to the FastAPI server in real-time. This web application enabled remote monitoring, accessing previous records, and visualization of the obtained images.

8.7 Cloud Performance Analysis

The cloud synchronization module was able to send the images captured, labels, the confidence score, and timestamps to the FastAPI server in real-time. This was achieved through the web dashboard that offered remote monitoring capability, data from previous times, and the display of images.

8.8 Challenges and Solutions

8.8.1 False Positives in Detection

Problem: Preliminary tests have shown that momentary or uncertain detections sometimes resulted in erroneous reactions from the hardware, especially with tool-shaped objects like pliers and wrenches, which look similar in X-rays.

Solution: The confirmation buffer design was carried out using a finite state machine of 12 frames, and it was such that the creation of an alarm would only be triggered after detection for 12 continuous frames.

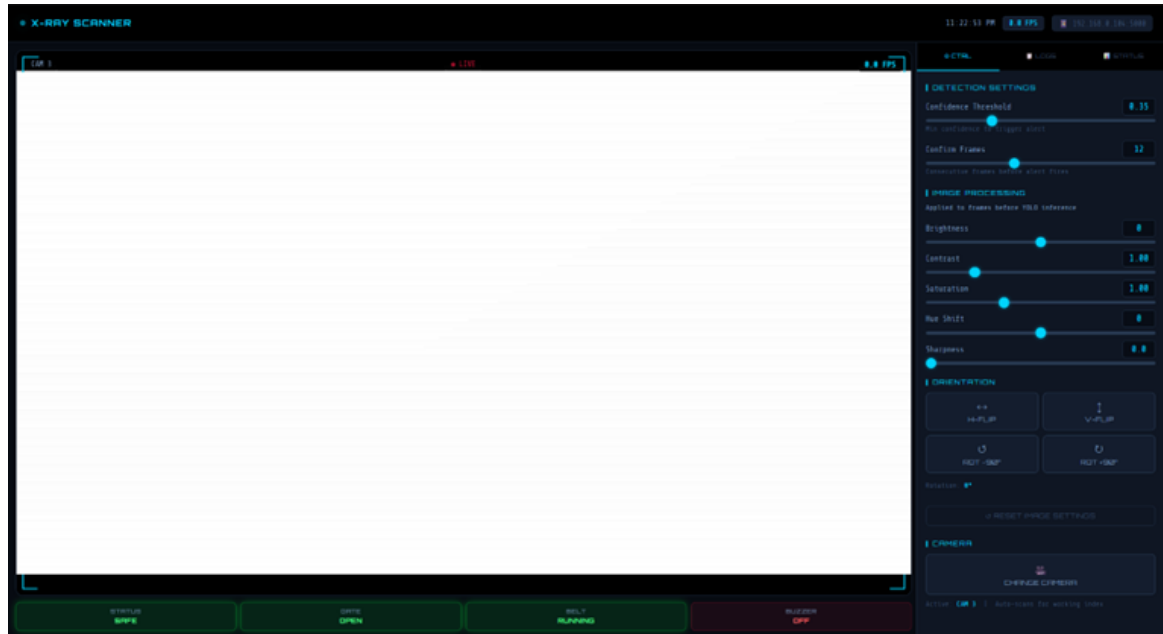


Figure 8.2: Cloud Dashboard Showing Safe State Detection



Figure 8.4: Cloud Dashboard Showing Danger State Detection

8.8.2 Inference Speed on CPU

Problem: CPU-based inference alone was found insufficient due to its poor performance, which was below the required 25 FPS limit.

Solution: Consequently, inference using CUDA and NVIDIA RTX 2060 GPU became the next step. Among models that were available for consideration, YOLOv8m was picked over other options such as YOLOv8l and YOLOv8x, which provided better results but at the cost of performance.

8.8.3 UART Communication Reliability

Problem: In initial testing for system integration, UART communication delays sometimes occurred between the detection unit and the STM32 microcontroller due to which the response from hardware was delayed..

Solution: To control the speed of sending UART commands, a command queuing process was introduced within the detection software program such that a delay was imposed between consecutive commands.

8.8.4 Object Overlap and Cluttered Luggage

Problem: However, the actual X-ray images of the luggage have many overlapping objects that may lead to the partial occlusion of the prohibited items.

Solution: In the training stage, several data augmentation methods such as mosaic augmentation, varying the brightness, and rotating were used. The new feature pyramid network used in the YOLOv8m architecture performs well in extracting features in different scales.

Chapter 9

Conclusion

In this case, we will introduce a smart suitcase threat detection system based on the technology of X-rays that combines the application of artificial intelligence with embedded automation and cloud monitoring. In this case, we will prove that just software-based detection is not enough to ensure successful deployment of our solution – automatic threat isolation is required.

Detection results achieved from the implementation of the YOLOv8m, utilizing a dataset of 8,295 images labeled on X-rays over 35 epochs of training, exhibited an average mAP of 0.91 at slightly less than 25 frames per second (FPS). Superior performance of the model in object detection is indicated by the extremely high value of mAP@50 achieved for all the objects detected.

The tasks involved in controlling the conveyor belt, solenoid-based object rejection, LEDs, buzzers, and LCD display were all accomplished using the STM32F103C8T6 microcontroller-based embedded automation system. An end-to-end delay of 150-200 ms is evidence that the system is feasible for practical applications using conveyors.

Intelligent integration based on cloud services through the application of FastAPI and PostgreSQL ensures centralization of remote monitoring, recording of incidents, and availability of archival data, hence providing a suitable solution for the scaling and execution of the multi-point solution. To conclude, the methodology described above deals with the relationship between smart identification and mitigation of threats, hence providing a practical and economical solution for current baggage screening requirements. Further research will focus on multi-camera integration, generalization of the model across different X-ray machines, and edge computing with GPU-based solutions.

Chapter 10

Future Scope

Even though the present project has been able to prove that artificial intelligence detection and embedded automation technologies can be successfully integrated in the domain of X-ray luggage screening, there are numerous other aspects wherein there exists room for improvement.

10.1 Edge AI Deployment

The existing system uses a standard laptop with NVIDIA RTX 2060 graphic card. Use of YOLOv8 on edge AI technologies such as NVIDIA Jetson Nano and NVIDIA Jetson Xavier will help to avoid using a central computer in the detection process. Such an approach will lower costs, make possible the use of multiple cameras per one checkpoint, and integrate detection hardware into the X-ray scanning structure.

10.2 Multi-Camera Integration

It will be important to increase the scale of operation of the data processing system in connection with the collection of data coming from multiple cameras viewing the conveyor belt from different angles. The use of multi-angle detection is expected to improve the likelihood of discovering objects that are hard to detect because of their position.

10.3 Exploration of Advanced Detection Architectures

Despite the fact that YOLOv8 is considered to provide an excellent balance between efficiency and effectiveness, future studies can include the testing of newer neural networks such as YOLOv9, YOLOv10, RT-DETR, and transformers, depending on their ability to identify threats from X-rays.

10.4 Realistic X-Ray Image Simulation

At present, the system utilizes a USB camera to capture the visual image of the objects put on the conveyor belt. However, once the system is integrated with actual hardware capable of producing X-ray images, it will be able to interpret actual X-ray images with distinct

features, including overlapping penetration, density-based color coding, and superimposing objects.

10.5 Enhanced Cloud Platform and Analytics

The current cloud backend solution is capable of logging events and capturing snapshots. Future enhancements might include employing artificial intelligence techniques to analyze trends such as popular object types, time periods during which threats are found, and operator performance assessment.

10.6 Improved False Positive Suppression

A future study could involve adaptive confirmation thresholds based on the conveyor speed, number of objects being examined, and the environment. The application of machine learning techniques to detect anomalies in the system will assist in differentiating between the threat and the non-threat object.

10.7 Scalability Through Cloud Deployment

The relocation of the backend server to the major cloud service providers such as AWS, Google Cloud, or Azure will ensure that scalability will be based on the occurrence of incidents detected, provide an opportunity for widespread implementation at different locations, and provide reliable backups for the data collected.

Bibliography

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.
- [2] G. Jocher et al., “Ultralytics YOLOv8 Documentation,” Ultralytics, 2023. [Online]. Available: <https://docs.ultralytics.com>
- [3] A. Krizhevsky, I. Sutskever, and G. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems*, vol. 25, 2012.
- [4] OpenCV Development Team, “Open Source Computer Vision Library,” 2023. [Online]. Available: <https://opencv.org>
- [5] STMicroelectronics, *STM32F103x8/xB Microcontroller Datasheet*, STMicroelectronics, Geneva, Switzerland, 2022.
- [6] M. Everingham et al., “The Pascal Visual Object Classes Challenge: A Retrospective,” *International Journal of Computer Vision*, vol. 111, no. 1, pp. 98–136, 2015.
- [7] T. Lin et al., “Microsoft COCO: Common Objects in Context,” *European Conference on Computer Vision (ECCV)*, 2014.
- [8] Roboflow Universe, “X-Ray Airport Prohibited Items Dataset.” [Online]. Available: <https://universe.roboflow.com>
- [9] FastAPI Documentation, “FastAPI — High Performance Web Framework for APIs,” 2023. [Online]. Available: <https://fastapi.tiangolo.com>
- [10] PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2023. [Online]. Available: <https://www.postgresql.org>
- [11] N. Dalal and B. Triggs, “Histograms of Oriented Gradients for Human Detection,” *IEEE Conference on Computer Vision and Pattern Recognition*, 2005.
- [12] S. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2011.
- [13] S. Akcay, M.E. Kundegorski, M. Devereux, and T.P. Breckon, “Transfer Learning Using Convolutional Neural Networks for Object Classification within X-Ray Baggage Security Imagery,” *IEEE International Conference on Image Processing (ICIP)*, 2016.

- [14] C. Miao, R. Xie, F. Wan, C. Su, H. Liu, J. Jiao, and Q. Ye, “SIXray: A Large-Scale Security Inspection X-Ray Benchmark for Prohibited Item Discovery in Overlapping Images,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [15] Y.F.A. Gaus, N. Bhowmik, S. Akcay, and T.P. Breckon, “Evaluating the Transferability and Adversarial Discrimination of Convolutional Neural Networks for Threat Object Detection within X-Ray Security Imagery,” *IEEE Transactions on Information Forensics and Security*, 2019.
- [16] C.Y. Wang, A. Bochkovskiy, and H.Y.M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [17] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” *arXiv preprint arXiv:1804.02767*, 2018.
- [18] P. Viola and M. Jones, “Robust Real-Time Face Detection,” *International Journal of Computer Vision*, vol. 57, no. 2, pp. 137–154, 2004.
- [19] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, “Internet of Things for Smart Cities,” *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, 2014.